FlowDesk 完整项目计划

1. 项目定义

项目名称：FlowDesk

定位：AI 驱动的个人 / 小团队工作管理平台。

一句话描述：

> 用户可以创建工作空间和项目，管理任务、成员、评论、文件与项目动态；AI Copilot 能理解工作空间中的数据，帮助拆解任务、总结项目、搜索信息，并在用户确认后执行部分操作。

典型使用场景包括个人 Side Project、软件开发项目、课程小组、比赛团队、自由职业项目以及小型创业团队。

整个产品围绕一个基本工作循环：

```
建立 Workspace
      ↓
创建 Project
      ↓
拆解 Task
      ↓
成员执行
      ↓
评论 / 文件 / 状态变化
      ↓
形成 Activity
      ↓
Dashboard 汇总
      ↓
AI 理解项目状态
      ↓
总结 / 搜索 / 建议 / 执行操作
```

第一版的目标不是挑战 Jira、Notion 或 Linear，而是完整实现这个闭环。

---

2. 项目最重要的学习目标

FlowDesk 有两个成功标准。

产品标准：

> 一个从未见过 FlowDesk 的用户，可以注册账号、创建 Workspace、邀请成员、创建项目和任务、上传附件、发表评论、查看 Dashboard，并使用 AI Copilot。

学习标准：

项目完成后，你能够解释：

```
Vue Component
      ↓
HTTP Request
      ↓
FastAPI Router
      ↓
Schema Validation
      ↓
Service
      ↓
ORM
      ↓
PostgreSQL
      ↓
Response
      ↓
Frontend State
      ↓
UI Update
```

以及：

```
User
 ↓
AI Copilot
 ↓
Backend
 ↓
收集 Project Context
 ↓
LLM API
 ↓
Tool Call
```

```
 ↓
Application Service
 ↓
Database
 ↓
Result
 ↓
LLM
 ↓
User
```

如果只是 Codex 帮你把这些全部生成出来，而你无法解释上述两条链，那么项目虽然“完成”，学习目标实际上没有完成。

---

3. 产品板块

整个 FlowDesk V1 划分为 **8 个板块**：

板块	核心作用	主要技术训练
Account	用户与认证	Auth、安全
Workspace	团队空间	数据建模、权限
Project	项目管理	CRUD、REST
Task	工作执行	SQL、复杂状态
Collaboration	评论/成员/活动	数据关系
Files	项目附件	文件系统/对象存储
Dashboard	项目分析	SQL 聚合/图表
AI Copilot	AI 辅助工作	LLM/Tool Calling

这八块就是项目的主体。

---

4. Account — 用户系统

第一阶段实现：

注册
登录
退出
保持登录状态

查看个人资料
修改个人资料
修改密码

User 数据大致包括：

```
User

id
username
email
password_hash
avatar_url
created_at
updated_at
```

这里不要急着使用第三方 Auth 服务把所有东西隐藏掉。

因为你应该亲自理解：

```
Password
   ↓
Hash
   ↓
Database

Login
   ↓
Verify Password
   ↓
Create Session / Token
   ↓
Cookie
   ↓
Authenticated Request
```

同时学习 Authentication 与 Authorization 的区别。

这是项目第一个重要工程模块。

---

5. Workspace — 工作空间

Workspace 是 FlowDesk 最大的业务边界。

例如：

```
Sched's Workspace

├─── FlowDesk
├─── Personal Website
└─── AI Research
```

或者：

```
Startup ABC

├─── Website
├─── Mobile App
├─── Marketing
└─── Operations
```

核心功能：

```
创建 Workspace
编辑 Workspace
删除 Workspace

邀请成员
移除成员
修改成员角色
```

初始权限：

```
Owner
Admin
Member
```

数据关系开始变得有意思：

```
User
  |
  |
Membership
  |
  ▼
Workspace
```

而不是：

```
User → Workspace
```

因为一个 User 可以属于多个 Workspace，一个 Workspace 又可以拥有多个 User。

你会第一次真正遇到：

Many-to-Many Relationship。

---

6. Project — 项目系统

Workspace 内创建 Project。

例如：

```
FlowDesk Development

Description
AI-powered project management platform

Status
Active

Start
2026-10-01

Deadline
2026-12-15
```

项目状态：

```
Planning
Active
Paused
Completed
Archived
```

Project 页面：

```
FlowDesk
```

```
Overview
Tasks
Files
Activity
Members
AI
Settings
```

这里主要学习：

REST API + CRUD + 数据验证 + 权限控制。

例如：

```
GET    /api/projects
POST   /api/projects
GET    /api/projects/{id}
PATCH  /api/projects/{id}
DELETE /api/projects/{id}
```

---

7. Task — FlowDesk 最核心的业务模块

Task 数据可以设计成：

```
Task

id
project_id

title
description

status
priority

assignee_id
creator_id

due_date

created_at
```

```
updated_at
completed_at
```

状态：

```
Todo
In Progress
Review
Done
```

优先级：

```
Low
Medium
High
Urgent
```

页面先做 List：

```
TASK              STATUS       PRIORITY   ASSIGNEE

Login API          In Progress   High       Alex
Register Page       Done         Medium     Sam
JWT Refresh         Todo         High       Alex
Fix Navbar          Review       Low        Lee
```

之后再增加 Kanban：

```
TODO         IN PROGRESS      REVIEW      DONE

JWT          Login API        Navbar      Register
Database      Dashboard
Email
```

拖动 Task：

```
Todo
 ↓
In Progress
```

实际上意味着：

```
Frontend Event
 ↓
PATCH /tasks/123
 ↓
```

```
Authorization
 ↓
Database UPDATE
 ↓
Activity Created
 ↓
Frontend Update
```

一个简单拖拽背后已经是一条完整工程链。

---

8. Collaboration — 协作系统

Task 可以发表评论：

```
Task #34

Implement Login API

Alex:
API 基本完成，但 refresh token 还没有实现。

Sam:
我明天处理前端错误状态。

Alex:
@Sam API 错误格式已经确定。
```

第一版：

发表评论
删除自己的评论
编辑评论

后期：

```
@Mention
Notification
Reaction
```

与此同时建立 Activity：

```
09:42 Alex created Task #34
```

```
10:13 Sam changed
Todo → In Progress

11:26 Alex commented

14:03 Sam uploaded auth-flow.png
```

这是后面 AI 理解：

“最近这个项目发生了什么？”

的重要数据来源。

---

9. Files — 文件系统

用户可以：

```
Upload File
Download File
Delete File
```

附件可以属于：

```
Workspace
Project
Task
```

第一阶段可以先存服务器本地：

```
/uploads/
```

等你理解文件上传后，再升级：

```
Application
    ↓
Object Storage
```

例如 S3-compatible storage。

这让你自然学习：

Multipart Upload、MIME、文件大小限制、权限、安全、对象存储。

不要第一天直接上复杂云存储。

---

10. Dashboard — 数据分析

Workspace Dashboard：

```
FlowDesk

Projects
12

Active
5

Tasks
————————————————————

Completed     128
In Progress    23
Todo           41
Overdue         7

Progress

████████████████▒▒▒▒ 72%
```

再增加：

```
Tasks Completed / Day

Mon  ████
Tue  ██████
Wed  ██
Thu  ████████
Fri  ████
```

以及：

```
Recent Activity
Upcoming Deadlines
Task Distribution
Project Progress
```

这一部分不是简单“画漂亮图”。

真正值得学习的是：

```
GROUP BY
COUNT
SUM
AVG
JOIN
ORDER BY
```

以及数据库查询效率。

---

11. Search — 建议加入的第九个能力

虽然不是独立大板块，但我建议 V1 后期加入全局搜索。

例如：

```
Search: authentication

Projects
FlowDesk Authentication Refactor

Tasks
#34 Implement authentication
#47 Fix authentication redirect

Comments
Alex: Authentication middleware...
```

第一阶段甚至可以：

```
ILIKE '%authentication%'
```

后面数据增加，再学习：

```
PostgreSQL Full Text Search
```

这样技术升级是由真实需求驱动的。

---

12. AI Copilot V1 — 最简单的 AI

AI 不应该在项目第一周出现。

等传统系统工作以后，再接模型 API。

第一项功能：

Task Generator

用户：

> 我要给 FlowDesk 添加忘记密码功能。

AI 返回：

```
Suggested Tasks

☑ Design password reset flow

☑ Create forgot-password endpoint

☑ Generate reset token

☑ Send reset email

☑ Create reset-password page

☑ Validate token expiration

☑ Add backend tests

[Create Selected Tasks]
```

注意 `[Create Selected Tasks]`。

这非常重要。

LLM 输出的不是 Markdown，而是类似：

```
{
  "tasks": [
    {
      "title": "...",
```

```
      "description": "...",
      "priority": "medium"
    }
  ]
}
```

你的程序解析以后显示 UI。

用户确认以后才写数据库。

这里学习：

Structured Output。

---

13. AI Copilot V2 — Context Engineering

下一阶段让 AI 理解项目。

用户：

总结 FlowDesk 目前的进展。

后端：

```
User Question
      ↓
Project Service
      ↓
查询

Project
Tasks
Deadlines
Recent Activity
Members

      ↓

构造 Context
      ↓
LLM
```

```
      ↓
Streaming Response
```

重点不是 Prompt 写得多漂亮。

而是：

哪些数据应该提供给模型？

例如 5,000 个 Task 不可能全部塞进去。

于是你开始面对：

```
Filtering
Aggregation
Context Window
Token Cost
Recency
Relevance
```

这是非常重要的 AI Engineering 能力。

---

14. AI Copilot V3 — Tool Calling

到这里才正式进入 Agent。

给 AI 一组工具：

```
get_project()

list_projects()

search_tasks()

get_task()

get_recent_activity()

get_project_statistics()
```

用户：

登录功能最近怎么样？

AI：

```
search_tasks("login")
      ↓
get_task(34)
      ↓
get_task(47)
      ↓
get_recent_activity()
      ↓
回答
```

你需要自己实现 Agent Loop：

```
LLM
 ↓
Tool Call
 ↓
Python Function
 ↓
Tool Result
 ↓
LLM
 ↓
Tool Call?
 ↓
...
 ↓
Final Answer
```

这里就是从：

AI Integration

进入：

Agent Engineering。

---

15. AI Copilot V4 — AI 执行操作

最后才允许：

```
create_task()
```

```
update_task()

assign_task()
```

例如：

把所有 authentication 相关的未完成任务优先级改成 High。

AI 找到：

```
#34
#47
#62
```

但**不要直接执行**。

界面：

```
AI wants to:

Change priority to HIGH

#34 Login API
#47 JWT Refresh
#62 Password Reset

[Cancel]       [Confirm]
```

用户确认：

```
Confirm
 ↓
Authorization
 ↓
Service
 ↓
Database
 ↓
Activity Log
```

这会让你第一次真正思考：

Agent 权限与安全。

这是比“AI 聊天框”高级很多的工程问题。

---

16. 推荐技术栈

V1 我建议：

```
Frontend
————————————————————
Vue 3
TypeScript
Vite
Vue Router
Pinia

Backend
————————————————————
Python
FastAPI
Pydantic
SQLAlchemy

Database
————————————————————
PostgreSQL

Migration
————————————————————
Alembic

AI
————————————————————
兼容 OpenAI-style API 的 LLM Provider

Infrastructure
————————————————————
Git
Docker
Docker Compose

Testing
————————————————————
Pytest
```

```
Vitest

CI/CD
──────────────────────────
GitHub Actions
```

CSS/UI 可以后续再决定，不需要现在纠结。

我也建议**暂时不用 LangChain**。

先亲自实现最基础的 LLM 调用、Streaming 和 Tool Loop。

---

17. 推荐项目结构

最终可能演化为：

```
flowdesk/

├── frontend/
│   ├── src/
│   │   ├── api/
│   │   ├── components/
│   │   ├── views/
│   │   ├── stores/
│   │   ├── router/
│   │   ├── types/
│   │   └── utils/
│   │
│   └── tests/
│
├── backend/
│   ├── app/
│   │   ├── api/
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── services/
│   │   ├── repositories/
│   │   ├── core/
│   │   ├── ai/
│   │   └── main.py
│   │
│   └── tests/
```

```
|
├── docs/
|
├── docker-compose.yml
├── .env.example
├── .gitignore
└── README.md
```

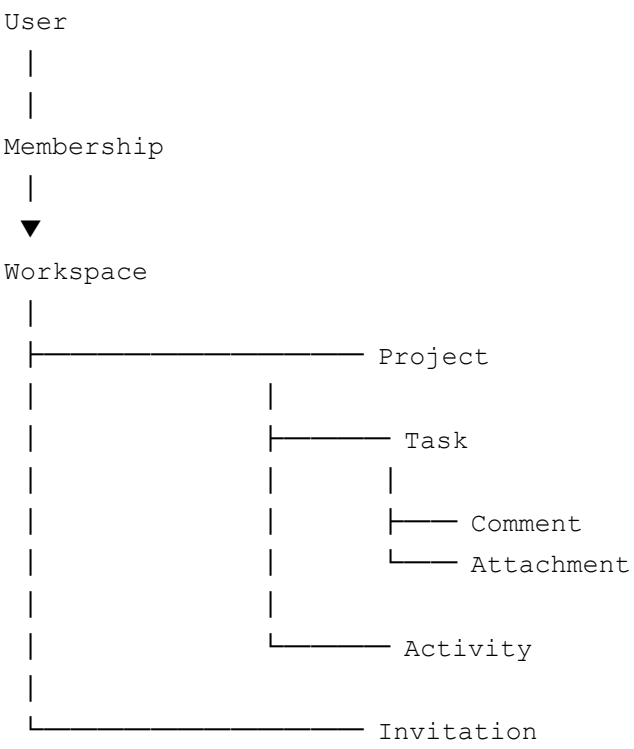
不过**第一天不要把所有空目录都创建出来。**

让架构随着项目逐渐成长。

---

18. 数据库核心模型

最终大致会形成：



主要表：

```
users
workspaces
workspace_members
invitations
```

```
projects

tasks
comments
attachments

activities

ai_conversations
ai_messages
```

不要一开始全部建立。

每开发一个功能再增加对应模型和 migration。

---

19. API 大致设计

最终可能有：

```
/api/auth/*
/api/users/*

/api/workspaces/*
/api/workspaces/{id}/members

/api/projects/*
/api/projects/{id}/tasks

/api/tasks/*
/api/tasks/{id}/comments
/api/tasks/{id}/attachments

/api/dashboard/*

/api/search

/api/ai/chat
/api/ai/generate-tasks
```

这样你会真正理解 REST API 的资源设计，而不是只会写：

```
/api/doSomething
```

---

20. 开发阶段：不要按“10 天”

我建议你完全放弃按天赶功能。

采用 **Milestone 制**。

Milestone 0 — Web 基础

目标：

```
Vue
 ↓
GET /api/health
 ↓
FastAPI
 ↓
{"status":"ok"}
 ↓
Vue 显示成功
```

只学习：

Git、Terminal、HTTP、JSON、Vue、FastAPI。

Milestone 1 — 第一个完整 CRUD

实现 Project：

```
Create
Read
Update
Delete
```

暂时甚至可以：

没有登录、没有 Workspace。

目标是第一次理解：

```
Vue → API → FastAPI → SQL → Database
```

Milestone 2 — Authentication

加入：

```
Register
Login
Logout
Authorization
```

Project 开始属于 User。

Milestone 3 — Workspace

加入：

```
Workspace
Membership
Roles
```

正式进入多用户数据模型。

Milestone 4 — Task System

实现：

```
Task CRUD
Status
Priority
Assignee
Deadline
```

然后 Kanban。

到这里 FlowDesk 已经是一个真正可用的软件。

Milestone 5 — Collaboration

加入：

```
Comments
Activity
Files
```

Milestone 6 — Dashboard & Search

加入：

```
Statistics
Charts
Search
```

到这里先停一下。

发布 FlowDesk 0.1。

没有 AI。

Milestone 7 — AI Integration

实现：

```
LLM API
Streaming
Structured Output
Task Generation
```

Milestone 8 — AI Context

实现：

```
Project Summary
Progress Analysis
```

Milestone 9 — Agent

实现：

```
Tool Calling
Search Tasks
Read Project
Read Activity
```

Milestone 10 — Production

最后处理：

```
Testing
Security
Logging
Docker
CI/CD
Deployment
Backup
Error Handling
```

发布：

FlowDesk 1.0

---

21. 时间尺度

不要把这个理解成严格 Deadline，但作为第一次开发，我会粗略预留：

阶段	大致学习 + 开发时间
M0	3 - 5 天
M1	5 - 8 天
M2	5 - 10 天
M3	5 - 8 天
M4	7 - 12 天
M5	5 - 10 天
M6	5 - 8 天
M7	4 - 7 天
M8	4 - 7 天

阶段 大致学习 + 开发时间

M9　7–14 天

M10　7–14 天

不是把这些数字机械相加。

随着能力提高，后面的开发速度通常会变快；另一方面，Debug 经常会让一个预计半天的问题拖两天。

所以我会把它看成一个大约 **2～3 个月的学习型项目**，而不是 10 天挑战。

---

22. Codex 和豆包 Agent 应该怎么用

你最大的风险不是项目太难，而是：

> **AI 太容易替你把项目做完。**

建议建立一个规则。

AI 可以大量帮助：

解释报错
Code Review
测试
查 API
解释源码
比较实现方案
发现 Bug
重构建议

谨慎让 AI：

一次生成完整模块

尽量不要：

“帮我把整个认证系统做好。”

更好的方式：

> “解释一个 Web 应用的登录请求从浏览器到数据库经历什么。不要写代码。”

理解以后：

“我们设计 FlowDesk 的 login endpoint，需要哪些输入输出？”

然后：

“我先实现，你帮我 review。”

如果不会：

“给我最小实现，并逐行解释。”

这会让 Codex 成为你的**高级 Pair Programmer**，而不是代写工具。

---

23. 项目版本规划

最终我建议形成三个正式版本。

FlowDesk 0.1 — Project Manager

```
Auth
Workspace
Project
Task
Comment
```

目标：证明你掌握基本全栈。

FlowDesk 0.5 — Collaboration Platform

```
+
Files
Activity
Dashboard
Search
Deployment
```

目标：证明你开始掌握工程开发。

FlowDesk 1.0 — AI Workspace

```
+
AI Task Generator
```

AI Project Summary
AI Search
Tool Calling
Agent Actions

目标：证明你能够把传统软件系统与现代 LLM 系统结合。

---

24. 暂时明确禁止加入的东西

项目过程中你很容易产生：

> “既然都做到这里了，不如再……”

然后项目爆炸。

所以 **1.0 之前不做**：

❌ 微服务
❌ Kubernetes
❌ 多 Agent
❌ 自己训练模型
❌ RAG / Vector DB
❌ 视频会议
❌ 在线文档编辑器
❌ Slack Clone
❌ Mobile App
❌ 复杂日历
❌ 支付系统
❌ 企业 SSO
❌ GitHub Code Agent

这些可以属于 FlowDesk 2.0。

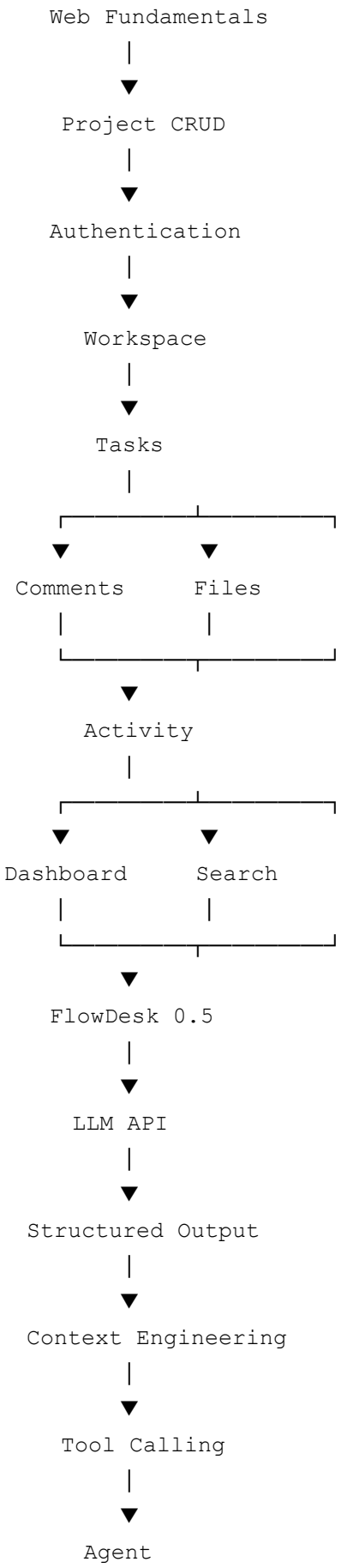
---

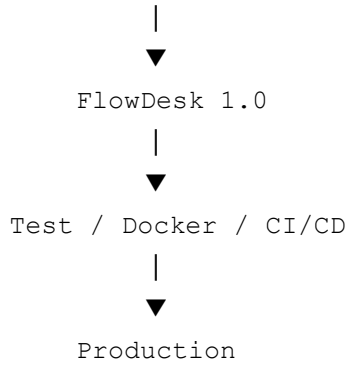
25. FlowDesk 真正的项目路线

最后把整个计划压缩成一张图：

FLOWDESK

|


Web Fundamentals
Project CRUD
Authentication
Workspace
Tasks
Comments
Files
Activity
Dashboard
Search
FlowDesk 0.5
LLM API
Structured Output
Context Engineering
Tool Calling
Agent



这才是我建议你正式采用的 FlowDesk 项目。

它的关键不是最终做出一个“项目管理网站”，而是你会按照 **Web 基础 → 全栈 → 数据库 → 多用户 → 权限 → 协作 → 工程化 → AI Integration → Agent** 的顺序走一遍现代软件开发链路。

而且有一个很重要的原则：**每完成一个 Milestone，都必须是可以运行、可以测试、可以提交 Git 的版本。** 即使你在 M7 决定停止，你前面的 FlowDesk 依然是完整的软件，而不是一个因为 AI 模块没做完就完全不能运行的半成品。

等真正开始开发时，第一件事情也不应该是让 Codex 创建 Vue/FastAPI 项目。应该先完成 **FlowDesk 0.1 的 PRD → 数据库 ER 图 → API 初稿 → 页面/用户流程 → Git 开发规范**。这五样确定以后，再写第一行正式业务代码。